```
'''1. Write a recursive function power(base, exponent) that, when
called, returns base^exponent
Example: power(3,4) = 3*3*3*3. Both base and exponent are user-
defined and assume that the
exponent is an integer greater than or equal to 1.
'''

def power(base,exponent):
    if(exponent==0):
        return 1
    return base*power(base,exponent-1)

print("Power 3^4:",power(3,4))
```

```
Power 3^4: 81
```

```
'''2. The greatest common divisor of integers x and y is the largest
integer that evenly divides into both x
and y. Write and test a recursive function gcd that returns the
greatest common divisor of x and y.
The gcd of x and y is defined recursively as follows: If y is equal
to 0, then gcd(x, y) is x; otherwise,
gcd(x, y) is gcd(y, x%y).'''

def gcd(x,y):
    if(y==0):
        return x
    return gcd(y,x%y)

print("GCD of 20 and 28:",gcd(20,28))
```

```
GCD of 20 and 28: 4
```

```
'''3. Write a recursive function that takes a number n as an input
parameter and prints n-digit strictly
increasing numbers.'''

def print_n_digits(n):
    if(n==0):
        return
    print_n_digits(n-1)
    print(n,end=" ")


print_n_digits(5)
```

```
1 2 3 4 5
```

```
'''4. Implement a recursive solution for computing the nth Fibonacci
number. Then, analyze its time
complexity. Propose a more efficient solution and compare the two
approaches.'''

#1st approach  Recusrsion T.C: O(2^n) S.C:O(n):
def nth_fibonacci(n):
```

```
    if(n==1):
        return 0
    if(n==2):
        return 1
    return nth_fibonacci(n-1)+nth_fibonacci(n-2)

#2nd approach  Recsursion+Memoization T.C: O(n) S.C:O(n):
def memo_nth_fibonacci(n,memo):
    if n in memo:
       return memo[n]
    if(n==1):
        return 0
    if(n==2):
        return 1
    memo[n]=memo_nth_fibonacci(n-1,memo)+memo_nth_fibonacci(n-
2,memo)
    return memo[n]
print(nth_fibonacci(6))
print(memo_nth_fibonacci(6,{}))
```

```
5
5
```

```
'''5. Given an array of N elements, not necessarily in ascending
order, devised an algorithm to find the
kth largest one. It should run in O(N) time on random inputs.'''
import random
def partition(arr, low, high):
    pivot = arr[high]
    i = low - 1

    for j in range(low, high):
        if arr[j] <= pivot:
            i += 1
            arr[i], arr[j] = arr[j], arr[i]

    arr[i + 1], arr[high] = arr[high], arr[i + 1]
    return i + 1

def quickselect(arr, low, high, k):
    if low == high:
        return arr[low]

    pivot_index = partition(arr, low, high)
    count = pivot_index - low + 1
    if count == k:
        return arr[pivot_index]
    elif count > k:
        return quickselect(arr, low, pivot_index - 1, k)
    else:
        return quickselect(arr, pivot_index + 1, high, k - count)

def find_kth_largest(arr, k):
    return quickselect(arr, 0, len(arr) - 1, len(arr) - k + 1)
```

```python
arr = [12,3,5,7,19,2,8]
k = 3
result = find_kth_largest(arr, k)
print(f"The {k}th largest element is: {result}")
```

```
The 3th largest element is: 8
```

```python
'''7. Given N points on a circle, centered at the origin, design an
algorithm that determines whether there
are two points that are antipodal, i.e., the line connecting the two
points goes through the origin. Your
algorithm should run in time proportional to NlogN.'''

import math

def are_antipodal_points(points):

    angles = []
    for (x, y) in points:
        angle = math.atan2(y, x)
        angles.append(angle)

    angles.sort()

    n = len(angles)

    for i in range(n):
        antipodal_angle = angles[i] + math.pi

        if antipodal_angle >= 2 * math.pi:
            antipodal_angle -= 2 * math.pi

        left, right = 0, n - 1
        found = False
        while left <= right:
            mid = (left + right) // 2
            if abs(angles[mid] - antipodal_angle) < 1e-9:
                found = True
                break
            elif angles[mid] < antipodal_angle:
                left = mid + 1
            else:
                right = mid - 1

        if found:
            return True

    return False

points = [(1, 0), (-1, 0), (0, 1), (0, -1)]
result = are_antipodal_points(points)
print("Antipodal points exist:", result)
```

```
Antipodal points exist: True
```

Q8)QUICK SORT

```
def quick_sort(arr):
    def quick_sort_helper(arr, low, high):
        if low < high:
            pivot_index = partition(arr, low, high)
            quick_sort_helper(arr, low, pivot_index - 1)
            quick_sort_helper(arr, pivot_index + 1, high)

    def partition(arr, low, high):
        pivot = arr[low]
        left = low + 1
        right = high

        while True:
            while left <= right and arr[left] <= pivot:
                left += 1
            while left <= right and arr[right] >= pivot:
                right -= 1

            if left > right:
                break
            else:
                arr[left], arr[right] = arr[right], arr[left]

        arr[low], arr[right] = arr[right], arr[low]
        return right

    quick_sort_helper(arr, 0, len(arr) - 1)


arr = [37, 2, 6, 4, 89, 8, 10, 12, 68, 45]
quick_sort(arr)
print("Sorted array:", arr)
```

```
Sorted array: [2, 4, 6, 8, 10, 12, 37, 45, 68, 89]
```

```
'''9. You are given the following list of famous personalities with
their net worth:
• Elon Musk: $433.9 Billion
• Jeff Bezos: $239.4 Billion
• Mark Zuckerberg: $211.8 Billion
• Larry Ellison: $204.6 Billion
• Bernard Arnault & Family: $181.3 Billion
• Larry Page: $161.4 Billion
Develop a program to sort the aforementioned details on the basis of
net worth using
a. Selection sort
b. Bubble sort
c. Insertion sort.
The final sorted data should be the same for all cases. After you
obtain the sorted data, present the
result in the form of the following dictionary:
{'name1':networth1, 'name2':networth2,...}'''
```

```python
##Selection Sort
def selection_sort(arr):
    n=len(arr)
    for i in range(n-1):
        min_idx=i
        for j in range(i+1,n):
            if arr[j][1]<arr[min_idx][1]:
                min_idx = j

        arr[i],arr[min_idx]=arr[min_idx],arr[i]

    return arr
def to_dict(arr):
    result = {name:networth for name,networth in arr}
    return result

def bubble_sort(arr):
    n=len(arr)
    for i in range(n):
        for j in range(n-i-1):
            if arr[j][1]>arr[j+1][1]:
                arr[j],arr[j+1] = arr[j+1],arr[j]

    return arr

def insertion_sort(arr):
    for i in range(1, len(arr)):
        key = arr[i]
        j = i - 1
        while j >= 0 and key[1] < arr[j][1]:
            arr[j + 1] = arr[j]
            j -= 1
        arr[j + 1] = key

    return arr

data=[("Elon Musk", 433.9),
      ("Jeff Bezos", 239.4),
      ("Mark Zuckerberg", 211.8),
      ("Larry Ellison", 204.6),
      ("Bernard Arnault & Family", 181.3),
      ("Larry Page", 161.4 )]
selection_sort_dict = to_dict(selection_sort(data.copy()))
print(f"Selection Sort:\n{selection_sort_dict}\n")

bubble_sort_dict = to_dict(bubble_sort(data.copy()))
print(f"Bubble Sort:\n{bubble_sort_dict}\n")

insertion_sort_dict = to_dict(insertion_sort(data.copy()))
print(f"Insertion Sort:\n{insertion_sort_dict}\n")
```

```
Selection Sort:
{'Larry Page': 161.4, 'Bernard Arnault & Family': 181.3, 'Larry
Ellison': 204.6, 'Mark Zuckerberg': 211.8, 'Jeff Bezos': 239.4,
```

```
'Elon Musk': 433.9}

Bubble Sort:
{'Larry Page': 161.4, 'Bernard Arnault & Family': 181.3, 'Larry
Ellison': 204.6, 'Mark Zuckerberg': 211.8, 'Jeff Bezos': 239.4,
'Elon Musk': 433.9}

Insertion Sort:
{'Larry Page': 161.4, 'Bernard Arnault & Family': 181.3, 'Larry
Ellison': 204.6, 'Mark Zuckerberg': 211.8, 'Jeff Bezos': 239.4,
'Elon Musk': 433.9}


'''10. Use Merge Sort to sort a list of strings alphabetically.
Example:
Input: ['apple', 'orange', 'banana', 'grape']
Output: ['apple', 'banana', 'grape', 'orange']'''
def merge(arr, left, mid, right):
    n1 = mid - left + 1
    n2 = right - mid
    L = [0] * n1
    R = [0] * n2
    for i in range(n1):
        L[i] = arr[left + i]
    for j in range(n2):
        R[j] = arr[mid + 1 + j]
    i = 0
    j = 0
    k = left
    while i < n1 and j < n2:
        if L[i] <= R[j]:
            arr[k] = L[i]
            i += 1
        else:
            arr[k] = R[j]
            j += 1
        k += 1
    while i < n1:
        arr[k] = L[i]
        i += 1
        k += 1
    while j < n2:
        arr[k] = R[j]
        j += 1
        k += 1


def merge_sort(arr,left,right):
  if left<right:
       mid = left + (right-left)//2
       merge_sort(arr,left,mid)
       merge_sort(arr,mid+1,right)
       merge(arr,left,mid,right)
  return arr
```

```python
arr = ['apple', 'orange', 'banana', 'grape']
print(f"Merge Sort:\n{merge_sort(arr,0,len(arr)-1)}")
```

```
Merge Sort:
['apple', 'banana', 'grape', 'orange']
```

```python
'''11. Without using the built-in sorted() function, write a Python program to merge two pre-sorted lists into
a single sorted list using the logic of Merge Sort. Example:
Input: [1, 3, 5, 7] and [2, 4, 6, 8]
Output: [1, 2, 3, 4, 5, 6, 7, 8]'''
def merge(l1,l2):
  n1 = len(l1)
  n2 = len(l2)
  arr = [0]*(n1+n2)
  p1 = 0
  p2 = 0
  k = 0

  while p1 < n1 and p2 < n2:
    if l1[p1] < l2[p2]:
      arr[k] = l1[p1]
      p1 += 1
    else:
      arr[k] = l2[p2]
      p2 += 1
    k += 1

  while p1 < n1:
    arr[k] = l1[p1]
    p1 += 1
    k += 1

  while p2 < n2:
    arr[k] = l2[p2]
    p2 += 1
    k += 1

  return arr

l1 = [1,3,5,7]
l2 = [2,4,6,8]
sorted_list = merge(l1,l2)
print(f"After merging sorted list:\n{sorted_list}")
```

```
After merging sorted list:
[1, 2, 3, 4, 5, 6, 7, 8]
```